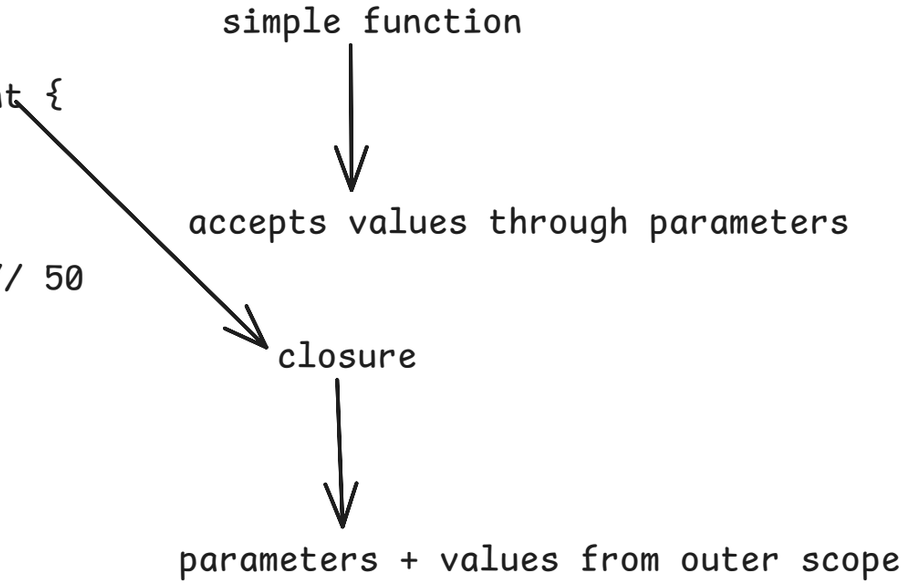
Lecture 05 - Interfaces and Generics

Functions

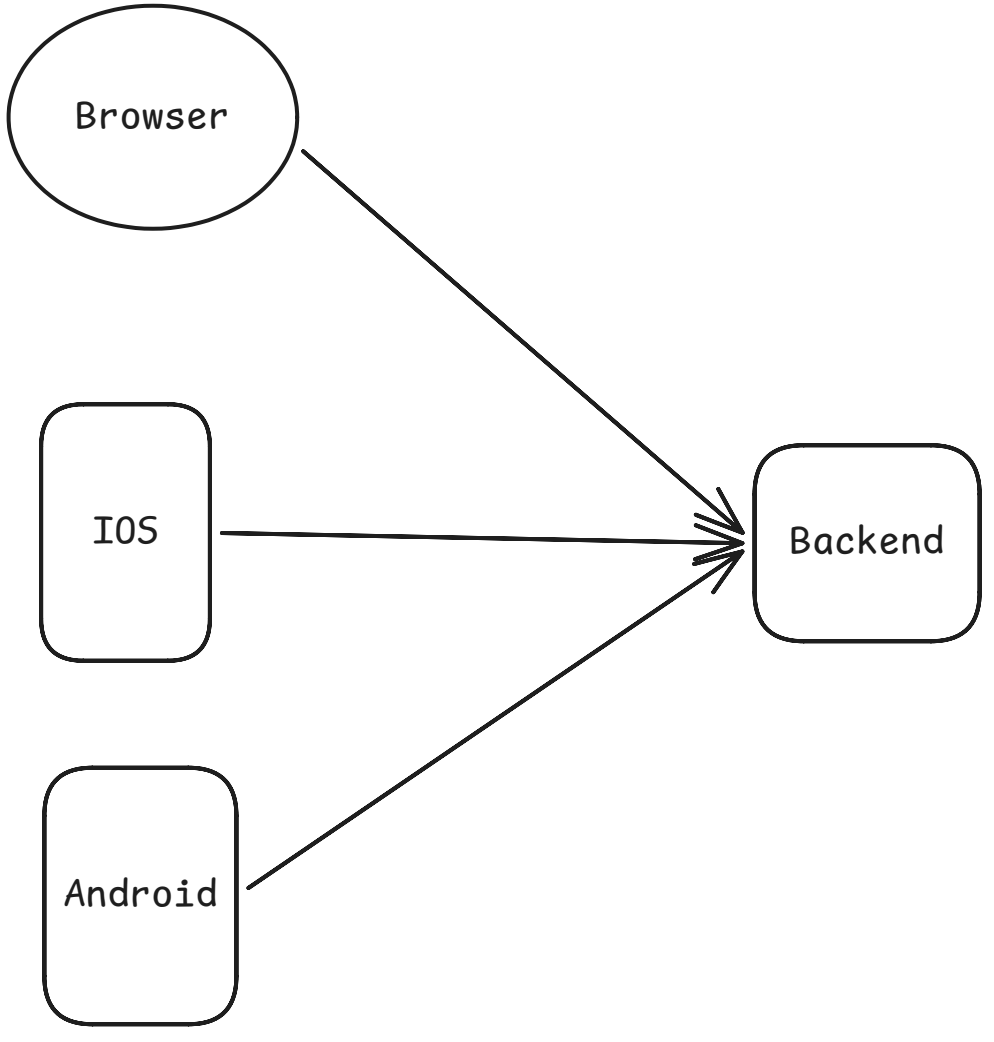
```
func divide(a, b float64) (float64, error) {

}

func main() {
    value := divide(10, 2)
    value, error := divide(10, 2)
}

file, error := os.Open()
```

```
func calculate(
    a, b int,
    operation func(int, int) int
) int {
    return operation(a, b)
}

func add(num1, num2 int) int {
    return num1 + num2
}

func main() {
    a := 10
    b := 10

    fmt.Println(calculate(a, b, add))
}
```

```
func main() {
    multiplier := 10
    multiply := func(x int) int {
        return x * multiplier
    }

    fmt.Println(multiply(5)) // 50
}
```



```go
func multiplier(factor int) func(int) int {
   return func(x int) int {
        x * factor
   }
}

double := multiplier(2)
triple := multiplier(3)
tenTimes := multiplier(10)

fmt.Println(double(4)) // 8
fmt.Println(triple(4)) // 12
fmt.Println(tenTimes(4)) // 40
```




```go
func minLength(min int) func(string) bool {
    return func(s string) bool {
        len(s) >= min
    }
}

password := minLength(8)
username := minLength(3)
description := minLength(10)
```

Errors as value

```
try:
    ...

except:
    ...
```

```
type error interface {
    Error() string
}

func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("can not divide by zero")
    }

    return a / b, nil
}



result, err := divide(5, 0)

if err != nil {
    // logs
    return fmt.Errorf("...: %w", err)
}

file, _ := os.Open("data.txt")

...

return fmt.Errorf("... %d", 5)
```

error.Is()

```
var ErrUserNotFound := erros.New("...")
```

user_service

```
func getUser() {
    user, err := userRepository.fetch(id: 123)

    if errors.Is(err, ErrUserNotFound)
        ...
    if errors.Is(err, ErrSqlConnection)
        ...
    if errors.Is(err, ErrSomeOtherProblems)
        ...
    error.Is()
}
```

```
try {

} catch (e UserNotFoundException) {

} catch (e SqlException) {

} ...
```

defer → execute this when surrounding function is about to return

```
func example() {
    // sql.Connection()

    defer sql.Close()
    defer fmt.Println("A")
    defer fmt.Println("B")

    // sql crashed

    // return err / panic

    // some db operation

    // cleanup the resources
    sql.Close()
}
```

```
func main() {
    x := 10

    defer fmt.Println(x) // 10

    x = 20
}
```

panic("...")